

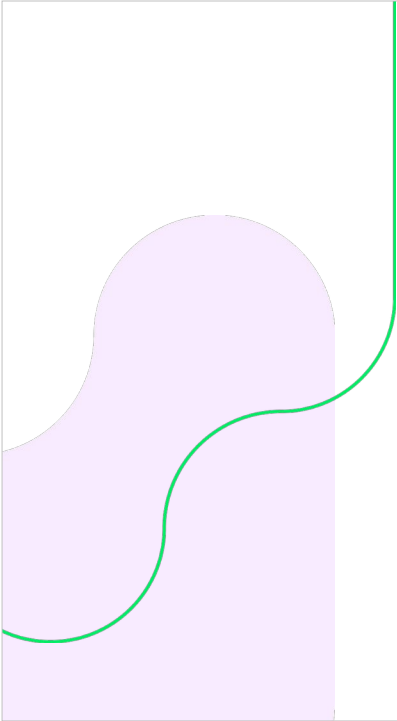


DA610

Advanced Queries and Data Processing

Developer Advanced Course

Release: 20260220



Topics we cover

Introduction and setup

Querying in MongoDB

Aggregation Stages and Expressions

- Summarization stages

- Connection stages

- Output stages

- Management stages

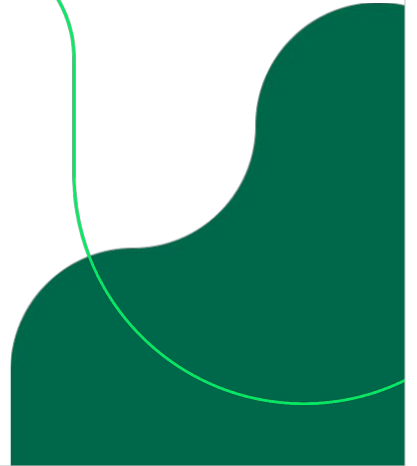
List Comprehension

Generating Queries and Pipelines in Code

Using \$let

Updates with aggregation pipelines

Introduction and Setup





Introduction

This workshop covers a wider range of aggregation pipeline concepts without being simply a list of stages to learn.

We focus on how to use aggregation in the shell and in your code.

We are not using the Aggregation builder in Compass or Atlas as these could limit the way you use the aggregation pipelines.



Setup

During this course we can use Atlas or a standalone MongoDB replica set:

- If you are using Atlas, create a replica set
- Or use your laptop with a installed MongoDB standalone (v4.2+)

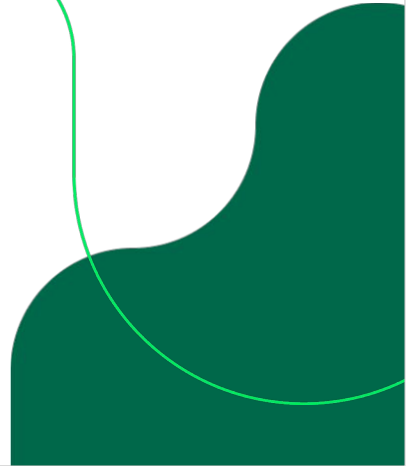
Download the Sample data sets:

- In Atlas, use the additional menu (...) and select **Download Sample Dataset**
- Or in your laptop, run this to load the data in your standalone replica set:

```
$ curl -OL https://mdb-tradecraft.s3-eu-west-1.amazonaws.com/m21_dump.tar.gz
$ tar xvf m21_dump.tar.gz
$ mongorestore
```

Use this Aggregations book for reference: <https://www.practical-mongodb-aggregations.com/>

Querying in MongoDB





Query Language concept

"Query languages or data query languages (DQLs) are computer languages used to make queries in databases and information systems."
(Wikipedia)

MQL (Mongo Query Language) does not have a specific language:

- Uses a model of specifying what data is required using **Objects** in the developer's choice of programming language
- MongoDB queries are intended to be written by **code**, not by humans



Selection and projection

Without Aggregation, MongoDB allows you to filter by row and field, and order results with a simple `find()` specifying three objects:

- filter
- projection
- sort

Exercise:

Write a query to return the **movie** released in **1933** that won the **most awards**. Use the **movies** collection in the **sample_mflix** database. Display only the **title** and **awards text**.



Exercise Solution

Exercise:

Write a query to return the movie released in 1933 that won the most awards. Use the **movies** collection in the **sample_mflix** database. Display only the title and awards text.

Solution:

```
db.movies.find({year:1933},{_id:0,"title":1,  
"awards.text":1}).sort({"awards.wins":-1}).limit(1)
```



Coding queries

- MongoDB commands are function calls
- Function parameters specify the query
- Each parameter is an Object
- Create the parameters dynamically

```
> db = db.getSiblingDB("sample_mflix")
> filter = {year:1933}
> projection = {_id:0,"title":1, "awards.wins":1}
> sortOrder = {"awards.wins":-1}
> doc =
db.movies.find(filter,projection).sort(sortOrder).limit(1).toArray()[0]
> nwins = doc.awards.wins
> filter2 = {year:1933,"awards.wins":nwins}
> projection2 = {_id:0,title:1,"awards.text":1}
> db.movies.find(filter2,projection2)
```

```
db = db.getSiblingDB("sample_mflix")
query = {year:1933}
projection = {_id:0,"title":1, "awards.wins":1}
sortOrder = {"awards.wins":-1}
doc =
db.movies.find(query,projection).sort(sortOrder).limit(1).toArray()
[0]
nwins = doc.awards.wins
query2 = {year:1933,"awards.wins":nwins}
projection2 = {_id:0,title:1,"awards.text":1}
db.movies.find(query2,projection2)
```



Querying Arrays in MongoDB

`$elemMatch` is an operator used to query arrays for an element that meets multiple criteria. The array must contain at least one element that matches the query.

Example:

```
record = { people : [
  { firstname: "john", lastname: "wayne" },
  { firstname: "jayne", lastname: "doe" }
] }
```

```
db.emp.insertOne(record)
```

```
query = {firstname: "john", lastname: "doe"}
```

```
db.emp.find({"people.firstname":"john","people.lastname":"doe"})
```

```
db.emp.find({people:{$elemMatch: query}})
```

When we are first shown `$elemMatch`, it is nearly always using an example where we return a false positive by querying on multiple fields. One of the array elements matches “firstname equals john”, and another one matches “lastname equals doe”.

Using `$elemMatch`, we can think of the query as being a “query by example”: so, find an *element* which exactly matches *this*. This is what happens in the command highlighted in green. However, it is important to understand that the argument to `$elemMatch` can be any query expression. So on arrays of Documents, like in the example, we can filter by equality, or by range, or by using relational operators like greater-than and less-than. You can take some time now to write a new query object, using relational operators to match the record.

This syntax works for arrays of documents, but if we want to query arrays of arrays, we need to take a different approach. We’ll look at this on the next slide.



Querying an array of arrays

Exercise:

Mean temperature of each week over the three months of summer (12 weeks)

```
r = [
  {mean_temps: [ [25,17,29,28], [26,18,22,28], [24,23,20,19] ]},
  {mean_temps: [ [15,17,29,22], [22,29,22,25], [25,27,33,25] ]}
]
db.temperatures.drop()
db.temperatures.insertMany(r)
```

Find a station reporting a mean weekly temperature of **greater than 32** in the **third week** of the **third month** of summer.

Solution:

```
db.temperatures.find({"mean_temps.2.2": {$gt: 32}})
```



Querying an array of arrays

Find a station reporting a mean weekly temperature of **greater than 32** in the **third week** of the **third month** of summer.

```
filter = {$gt: 32}
db.temperatures.find({"mean_temps.2.2": filter})
```

```
{mean_temps: [ [15,17,29,22], [22,29,22,25], [25,27,33,25] ]}
```

Find **all** stations where a temperature average was above 32?

```
db.temperatures.find({"mean_temps": filter})
db.temperatures.find({"mean_temps.2": filter})
```

You may have come up with a solution like this. We can specify distinct elements in an array using the dot notation, so we filter on the field “mean_temps dot 2 dot 2”. This works and allows us to specify a specific element in a nested array. The ‘dot notation’ is also known as the ‘positional operator’.

However, what if we wanted to find a station reporting a mean temperature of greater than 32 in *any* week of *any* month? Here the positional operator can’t help us. If we apply the filter on the array of arrays itself, we will find no matches, and even if we apply the filter on a specific element in the array, we would also find no matches.



\$elemMatch with array of arrays

Example - Solution:

Mean temperature of each week over the three months of summer (12 weeks):

```
{ mean_temps: [ [25,17,29,28],[26,18,22,28],[24,23,20,19] ] },  
{ mean_temps: [ [15,17,29,22],[22,29,22,25],[25,27,33,25] ] }
```

Find **all** stations where the temperature average was over 32:

```
filter = {$gt: 32}  
nested_array_filter = {$elemMatch: {$elemMatch: filter} }  
mean_temps_filter = { mean_temps: nested_array_filter}  
db.temperatures.find(mean_temps_filter)
```

The solution is to use nested \$elemMatch commands. Remember that the parameter that \$elemMatch takes is a query expression. The thing we are interested in, an element greater than 32, is our query expression. We then use \$elemMatch twice to match the level of nesting.

```
query = {$gt: 32}  
query_temp = {$elemMatch: {$elemMatch: query} }
```

Then we use that query expression to look for a document where the mean_temps array contains a value greater than 32.

When thinking about how to query something - especially if there are arrays involved - do not rule out \$elemMatch initially - it's more powerful than you expect



\$elemMatch with array of arrays

Exercise:

Mean temperature of each week over the three months of summer (12 weeks)

```
r = [{ mean_temps: [ [25,17,29,28], [26,18,22,28],  
                    [24,23,20,19] ] },  
      { mean_temps: [ [15,17,29,22], [22,29,22,25],  
                    [25,27,33,25] ] }]  
db.temperatures.drop()  
db.temperatures.insertMany(r)
```

1. Find records (from the ones above) where there was a week with an average greater than 33
2. Find where we had a week with an average temperature of 23 exactly

Answers in page 9

1. Find records (from the ones above) where there was a week with an average greater than 33

```
query = { mean_temps: { $elemMatch : { $elemMatch : { $gt :  
33}}}}}  
db.temperatures.find(query)
```
2. Find where we had a week with an average temperature of 23 exactly

```
query = { mean_temps: { $elemMatch : { $elemMatch : { $eq :  
23}}}}}  
db.temperatures.find(query)
```



Expressive Queries

You can apply some basic computation to the values in a record, transforming, or comparing them to each other.

If the result of the expression is **true** then the record is returned.

Example:

```
db.movies.find({$expr : true})    //This returns all movies
```

`$expr` evaluates the expression for each record

Can use simple aggregation expressions like “`$fieldname`”

From MongoDB 3.6 onwards, the query engine has access to aggregation expressions - we will talk about Aggregation expression later.

You can apply some basic computation to the values in a record, notable transforming, or comparing them to each other if the result of this is the boolean value true then the record is returned we can see this with a very simple query which returns everything.



Expressive Queries - Exercise

Understand and run the following:

```
db = db.getSiblingDB("sample_mflix")
hasHighRating = {"imdb.rating": {$gt:9}}
flagAsGood = {$set: { awesome : true}}

db.movies.updateMany(hasHighRating, flagAsGood)
```

Find the movies with the awesome flag using \$expr

Solution:

```
findAwesome = { $expr : "$awesome" }
db.movies.find(findAwesome)
```



Expressive Queries

In a similar way we can use an aggregation expression directly:

```
condition = { $gt : [ "$imdb.rating", 9 ] }  
simpleQueryExpr = { $expr : condition }
```

```
db.movies.find(simpleQueryExpr)
```

Or \$cond

```
redundantQueryExpr = simpleQueryExpr = { $expr : { $cond : { if:  
condition, then: true, else: false } } }
```

```
db.movies.find(redundantQueryExpr)
```

Why do the following queries return different results?

```
db.movies.find({"imdb.rating":{"$gt:9}}).count()  
db.movies.find({$expr: { $gt: ["$imdb.rating",9]}}).count()
```

Answer:

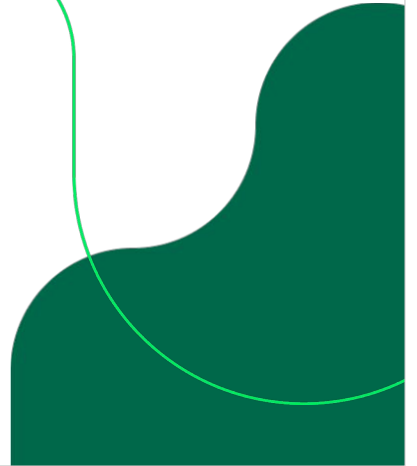
This is due to the way "" is compared to 9 - this is an important thing to note.

MongoDB has semantics for ordering different types when sorting but apart from numbers does not compare different types. This is called type bracketing.

In aggregation we do take the ordering of types into account so $null < 5 < ""$ for example. In aggregation type bracketing does not happen.

This can catch you out where you have nulls although treating $null < 0$ may make sense.

Aggregation Pipelines

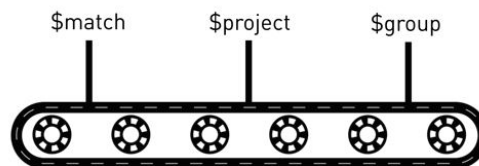




Aggregation Pipelines

Aggregation pipelines allow you specify a number of stages, each of which reads a stream of incoming documents and outputs those documents transformed in some way.

Each transformation is a single step known as a stage



Aggregation pipelines allow you specify a number of stages, each of which reads a stream of incoming documents and outputs those documents transformed in some way, the stream may be shortened by filtering or grouping, reordered by a sort or lengthened via an unwind.



Compared to SQL

MongoDB **find()**:

```
SELECT a,c,b  
FROM database.table  
WHERE d<100  
ORDER BY d ASC
```

MongoDB **aggregate()**:

```
SELECT b+c as a,SUM(e) AS t  
FROM D.T LEFT JOIN D.T2 ON y  
WHERE T2.A=T.B  
GROUP BY a  
HAVING t > 100
```

MongoDB's find is equivalent of SQL SELECT and WHERE statements

Aggregate stages such as \$GROUP and \$LOOKUP can be equivalent of SQL join and group

Find() and Aggregate are slowly converging in MongoDB so this distinction gets less and less with each release.



Simple Aggregation Stages

<code>\$match</code>	equivalent to <code>find(query)</code>
<code>\$project</code>	equivalent to <code>find({},projection)</code>
<code>\$sort</code>	equivalent to <code>find().sort(order)</code>
<code>\$limit</code>	equivalent to <code>find().limit(num)</code>
<code>\$skip</code>	equivalent to <code>find().skip(num)</code>
<code>\$count</code>	equivalent to <code>find().count()</code>

When using these at the start of a pipeline, the query optimizer transforms them to a `find()`

The most common stages are `$match`, `$project`, `$sort`, `$limit`, `$skip`, `$count` and `$group`. We will assume you are already familiar with the basic concepts of a pipeline using `$match`, `$sort`, `$project` - as you would in a `find()` as well as a simple example of `$group` or `$unwind` but let's test that with some basic exercises.

You may need to refer to the online documentation to complete these.



Aggregation Syntax Example

Use **aggregate()**:

```
db.listingsAndReviews.aggregate([
  {$match:{"address.country":"Canada"}},
  {$sort:{"host.host_total_listings_count":-1 }},
  {$limit:1},
  {$project:{
    "host.host_total_listings_count":1,
    "host.host_name":1}}
])
```

This is an example of the Aggregation syntax



Exercise: Simple Aggregation Stages

In *sample_mflix* there is a *movies* collection. The *year* field contains the year the film was released. The *countries* array contains a list of the countries that the film was released in.

From all the documents in this collection, determine when the first film was ever released in each country and order results to show the latest year first.

Format example: { country: "Egypt", year:1960 }

Solution:

```
unwind = { $unwind : "$countries" }
group = { $group : { _id: "$countries", year: {$min:"$year"}} }
project = { $project: {country : "$_id",year:1, _id: 0}}
sort = { $sort : {year:-1}}
db.movies.aggregate([unwind,group,sort,project])
```



Simple Aggregation Expressions

- Expressions return one or more fields in the current document along with transformation of those fields
- The expression "\$fieldname" returns the content of the field
- Expressions are written as follows:

```
{ $transformName: expression }
```



```
{ $transformName : [ expression1, expression2, ... ] }
```
- See "[Expressions](#)" in the official documentation

Aggregations also make extensive use of expressions.

Expressions return one or more fields in the current document along with transformation of those fields - the simplest expression is "\$fieldname" which is just the content of the field - however there are many transformative expressions.

Expressions are written as `{ $transformName: expression }` or `{ $transformName : [ expression1, expression2, ... ] }` depending on whether it takes one or more than one input parameters. They always use an array for multiple 'arguments'

See <https://www.mongodb.com/docs/manual/reference/mql/expressions/> for a full list



Aggregation Expressions Example

Execute:

```
r = { name: "John Page", awards: 22, merits : 5, startdate: ISODate("2013-08-02")}
```

```
db.example.drop()
```

```
db.example.insertOne(r)
```

```
db.example.aggregate({ $project : { v : "$name" } })
```

```
db.example.aggregate({ $project : { v : { $substr: ["$name", 0, 1] } } })
```

```
db.example.aggregate({ $project : { v : { $split: ["$name", " "] } } })
```

```
db.example.aggregate({ $project : { v : { $sum : ["$awards", "$merits"] } } })
```

```
db.example.aggregate({ $project : { v : { $avg : ["$awards", "$merits"] } } })
```

```
//Number of milliseconds
```

```
now = new Date()
```

```
db.example.aggregate({ $project : { v : { $subtract : [now, "$startdate"] } } })
```

Understand and execute:

```
r = { name: "John Page", awards: 22, merits : 5, startdate:
ISODate("2013-08-02")}
```

```
db.example.drop()
```

```
db.example.insertOne(r)
```

```
db.example.aggregate({ $project : { v : "$name" } })
```

```
db.example.aggregate({ $project : { v : { $substr: ["$name", 0, 1] } } })
```

```
db.example.aggregate({ $project : { v : { $split: ["$name", " "] } } })
```

```
db.example.aggregate({ $project : { v : { $sum :
["$awards", "$merits"] } } })
```

```
db.example.aggregate({ $project : { v : { $avg :
["$awards", "$merits"] } } })
```

```
//Number of milliseconds
```

```
now = new Date()
```

```
db.example.aggregate({ $project : { v : { $subtract :
[now, "$startdate"] } } })
```



Aggregation Stage Categories

We'll be looking at how to reshape, summarise, connect and output data using various categories of aggregation stages.



Aggregation Reshape Stages

- `$project`
- `$set ($addFields)`
- `$redact`
- `$replaceRoot`

Aggregation has four main ways to reshape a document:

\$project describes exactly what the documents should look like after this stage. It is however an absolute description rather than a relative one. A `$project` is not required as a performance/memory optimisation either as the pipeline can figure out for itself the minimum set of required data.

\$set is used to add an individual, or multiple fields each of which is defined by an expression. Previously `$set` was more commonly used with its alias `$addFields`, both still are available.

- You can also remove items via conditional expressions within a `$project` or `$set` by setting a field to `$$REMOVE` as part of the evaluated boolean expression.
- The `$unset` stage is an alias and simpler format of the `$project` stage that removes or excludes fields. Note that the behavior of `$unset` in an update operation is different from `$unset` in an aggregation pipeline.

\$redact is used to traverse object structures as a hierarchy, it has very limited utility.

\$replaceRoot makes it easy to take any object you create from an expression and use it as the whole document in the next stage..



Aggregation Reshape Stages - Exercises:

Write two aggregation pipelines to meet these requirements:

- In the `sample_training.grades` return only the elements within the `scores` array where the `type` is `exam`
- In the `sample_mflix.movies` remove all mention of `awards` (even the fieldname) if `Will Smith` is in the `cast`

Exercises from Workbook (page 14)

Exercise 1 - Solution:

```
use sample_training

match = {$match: {"scores.type": "exam"}}
projection = {
  $project: {
    exams_scores: {
      $filter: {
        input: "$scores",
        cond: {
          $eq: ["$$this.type", "exam"]
        }
      }
    }
  }
}

pipe = [match, projection]
db.grades.aggregate(pipe)
```

Exercise 2 - Solution:

```
use sample_mflix
condition = {$in: ["Will Smith", "$cast"]}
pipe = [{ $set : { awards: { $cond: [condition, "$$REMOVE", "$awards"] }}}]
db.movies.aggregate(pipe);
```



Summarization Stages

Aggregation requires BSON data to be **processed** into internal objects and then back into BSON

Summarization stages **reduce the amount of data** processed

- `$group`
- `$bucket`
- `$bucketAuto`
- `$sortByCount`
- `$facet`

It is important to understand the overheads of using Aggregation, where the BSON data must be processed into internal objects by the server and converted back to BSON as results (except in a case where the whole pipeline can be expressed as a query).

Because of this - a pipeline that returns a large quantity of the data in the database or a large portion of the records touched may not make sense, it may be faster and simpler to simply return the raw data for processing at the driver end.

For aggregation to be truly useful it must be able to perform significant processing and aggregation of data at the server end, to return a summarized view of the information saving the cost in CPU and network of returning the information itself. We do this using the summarisation stages - ones which reduce documents to a smaller set of documents reducing the fidelity/resolution of the information.

Aggregation Summarization stages:

- `$group` - create one output document for each unique value of a given expression in the input documents (aggregate functions such as `$sum` and `$avg` are often used). Example: group cities by US state.
- `$bucket` - create one output document for each supplied range of values, grouping together documents where a given expression falls into that range. For example, group people's cars by engine capacity rounded to the nearest 500.
- `$bucketAuto` - create N output documents of equal size by automatically determining what range of value to place in each group. This includes many variations on this to choose how these divisions are calculated.
- `$sortByCount` - create a count of how many of each value we have for a given expression and return the counts, most common first.
- `$facet` - Create a single document containing the results of one or more pipelines based on the input data.



Summarization Stages - Exercise 1

Use the `sample_mflix` dataset to write pipelines and answer the following:
How many movies there are per **decade** - for example 1890-1899?
Show results ordered by decade

- Do this in two ways using `$group` and `$bucket`.
- Can you do it using `$bucketAuto` too?

Exercise (Workbook page 16 & 17):

Solution using `$group`:

```
db=db.getSiblingDB("sample_mflix")

fixyear = {$set:{year:{$convert:
{input:"$year",to:"int",onError:0,onNull:0}}}}
groupbydecade = { $group : { _id : {$multiply : [ 10, { $floor
: { $divide : [ "$year",10] }}] }, count : {$sum:1}}}
sortbydecade = { $sort : {_id:1}}

db.movies.aggregate([fixyear,groupbydecade,sortbydecade])
```

Solution using `$bucket`: You compute dynamically the boundaries in the shell to pass to the aggregation

```
db=db.getSiblingDB("sample_mflix")
db.movies.createIndex({year:1})
oldest = db.movies.find({year:
{$type:"number"}}).sort({year:1}).limit(1).next().year
latest=db.movies.find({year:
{$type:"number"}}).sort({year:-1}).limit(1).next().year

buckets=[]
for(y=oldest - (oldest%10);y<latest+10;y=y+10){
  buckets.push(y);
}

fixyear = {$set:{year:{$convert:
{input:"$year",to:"int",onError:0,onNull:0}}}}
intobuckets = {$bucket : { groupBy: "$year", boundaries:
buckets, default:0, output: {count:{$sum:1}}}}
```



Summarization Stages - Exercise 2

Follow the previous exercise:

Use `$group` or similar to get the **list of countries** and their corresponding earliest release years and find the **midpoint** in that list.

Try to achieve this with a couple more stages and some expressions:

<https://docs.mongodb.com/manual/meta/aggregation-quick-reference/#expressions>

Exercise (Workbook page 18):

The Output you are trying to get is: `{ "country" : "Czech Republic", "year" : 1992 }` or any other Country with 1992

Solution:

```
db = db.getSiblingDB("sample_mflix")

stage1 = { $unwind : "$countries" }
stage2 = { $group : { _id:"$countries",year:{$min:"$year"}}}
stage3 = { $project: { country : "$_id",year:1}}
stage4 = { $sort : {year:-1}}
stage5 = { $group : { _id: null, countries:{$push:
{country:"$country",year:"$year"}}}}
stage6 = { $project : { median : {$arrayElemAt :
[ "$countries", {$divide:[{$size:"$countries"},2]]}}}
stage7 = { $replaceRoot : { newRoot : "$median"}}
db.movies.aggregate([stage1,stage2,stage3,stage4,stage5,stage6,
stage7])
```



Connection Stages

`$lookup`

- Pulls data into the pipeline from a second collection
- Equivalent to a left outer join
- Only rapidly changing dimensions should be looked up
- Excessive use of this stage will negatively affect performance
- Has two modes: simple version and a more sophisticated version (Expressive `$lookup`)

Connection stages:

`$lookup`

During aggregation MongoDB can pull data into the pipeline from a second or subsequent collection.

This is done using `$lookup` - whilst this is equivalent to a left outer join (always include the row from the table on the left) it is important to consider the name `$lookup` as that gives information about what it is intended to be used for.

Document Schema design says you should embed information that is a direct child 1:1 or 1:N or any N:M lookup value that does not change very often. Only rapidly changing dimensions should be looked up.

`$lookup` has two modes:

- The simple version exactly matches a field in the source document to a field in one or more other documents and copies the contents of them in, it does this by running queries on the server to find matching documents.
- The more sophisticated version runs a different aggregation pipeline and includes the output of that as an array of documents in the source.



Connection Stages - Exercise:

Update the `movies` database and create a new version where for each movie we can see all the other movies by the same director - to seed a 'you might also like' page.

Use a simple `$lookup` to `add` a field in each movie called `seeAlso` that includes the director's other movies.

Use a `$set` after the `$lookup` to project just the titles of these other movies.

Exercise (Workbook page 19):

Part 1- execute aggregation with `$lookup` stage

```
db=db.getSiblingDB("sample_mflix")
lookupOthersByDirectors = { $lookup : {from: "movies",
localField: "directors", foreignField: "directors", as:
"seeAlso" }}
movieTitlesOnly = {$set : {seeAlso : "$seeAlso.title"}}
db.movies.aggregate([lookupOthersByDirectors, movieTitlesOnly])
```

Part 2- measure how long it takes. This is surprisingly slow , run the following to see how long it takes :

```
var start=new Date()
db.movies.aggregate([{$limit:1000},lookupOthersByDirectors,movieTitlesOnly,{$count:"totaldocs"}])
var timeTaken=(new Date() - start)/1000
print(`took ${timeTaken} seconds`)
```

Part 3- Make it faster. That's too long , it normally takes about 20 seconds - can you fix it , why was it slow? Solution: There is no an index. Create one and try again:

```
db.movies.createIndex({directors: 1})

//Execute again step 2
var start=new Date()
db.movies.aggregate([{$limit:1000},lookupOthersByDirectors,movieTitlesOnly,{$count:"totaldocs"}])
var timeTaken=(new Date() - start)/1000
print(`took ${timeTaken} seconds`)
```



Expressive \$lookup

- A simple \$lookup is very limited
- If we need to **summarise data** or query on multiple fields, we use an expressive lookup

Example:

Rank directors by number of awards
Add that director ranking to each movie.

```
addRank = { $lookup: { from: "movies", let: { directors: "$directors" }, pipeline: dirRankPipeline, as: "rank" }}
```

A simple \$lookup is very limited - it can pull in whole documents based on a single exact matching field - it cannot summarise data or query on multiple fields - if we want to do more we need to use an expressive lookup.

Example:

In order to rank a director compared to all the rest, you need to:

- 1) work out a pipeline to rank a director based on the awards they have received. We also want to deal with the fact that movies have multiple directors which the previous \$lookup didn't handle.
- 2) Read through the following that computes the relative ranking of each director. You cannot compute the rank of a Director without comparing to all the other directors so we can't just do this for one director with a match at the start.

Note: This solution is amazingly slow if we apply it to the whole data set. You can time it (see note below)

Solution:

```
unwindDirectors = { $unwind : "$directors" }
awardsPerDirector = { $group : { _id: "$directors", awards:
  { $sum: "$awards.wins" } } }
orderByAwards = { $sort: { awards: -1 } }
groupToOneDoc = { $group: { _id: null, name: { $push: "$_id" } } }
unwindWithIndex = { $unwind:
  { path: "$name", includeArrayIndex: "directorRank" } }
dontStartAtZero = { $set: { _id: "$$REMOVE", directorRank: { $sum: [1,
  { $toInt: "$directorRank" } ] } } }

db=db.getSiblingDB("sample_mflix")
dirRankPipeline=[unwindDirectors,awardsPerDirector,orderByAwards,group
ToOneDoc,unwindWithIndex,dontStartAtZero]
db.movies.aggregate(dirRankPipeline)
```



Output Stages

\$out	\$merge
● Cannot update an existing collection ● The output collection will be replaced with the aggregation pipeline output ● If the target collection already exists it will keep existing indexes	● Allows to update existing collections ● Can be used for maintaining on demand materialized views
The output collection cannot be sharded	The output collection can be sharded

The aggregation framework has the ability to redirect the output of an aggregation pipeline into a collection using \$out or \$merge stages.

Here the main differences between the two stages.



Output Stages - Exercise 1:

Considering the previous aggregation sub pipeline we built with director rankings from the movies collection and save the output using \$out

Create a new collection with the director ranks in:

```
db.movies.aggregate([addRank, {$out: "rankDirector"}]) //This fails
```

```
db.movies.aggregate([
  {$match : { directors : { $type: "array" } }},
  dirRankPipeline.pop().flat(),
  {$out: "rankDirector"}])
```

Use this collection in the \$lookup of the previous activity

Exercise (Workbook page 23):

Using \$out

```
db.movies.aggregate([addRank, {$out: "rankDirector"}])
//This fails as some documents do not have directors. Try this
instead:
```

```
db.movies.aggregate([{$match: {directors: {$exists: true}}},
  dirRankPipeline, {$out: "rankDirector"}])
```

```
db.rankDirector.createIndex({name:1})
db=db.getSiblingDB("sample_mflix")
```

```
lookupRankDirector = { $lookup : {from: "rankDirector",
  localField: "directors",
  foreignField: "name",
  as: "director_rankings" }}
```

```
db.movies.aggregate([lookupRankDirector])
```

Compare speed with the expressive lookup



Output Stages - Exercise 2:

An IoT device is collecting sensor values and storing the data in a collection, but the raw data has too high resolution for showing it in a chart efficiently.

The sensor values are stored in bucket documents, with a size of 10.

Use `$merge` to create an additional collection called `aggregatedValues` to aggregate the raw data calculating the mean values for each bucket.

In mongo shell, run simulate a temperature and humidity sensor using the code provided by the instructor.

Exercise (Workbook page 24 & 25):

- 1) Provide this code to Simulate the sensor data collection:

```
db = db.getSiblingDB("measurements");
db.sensorValues.createIndex({ count: 1 });
db.sensorValues.createIndex({ startTime: 1 });
var bucketSize = 10;

function simulateSensor() {
  while (true) {
    var temp = 10 + Math.random() * 20;
    var humidity = 10 + Math.random() * 90;
    db.sensorValues.updateOne(
      {
        count: { $lt: bucketSize }
      },
      {
        $push: { values: { "timestamp": new
Date(), temp: temp, humidity: humidity } },
        $min: { startTime: new Date(), tempMin:
temp, humidityMin: humidity },
        $max: { endTime: new Date(), tempMax:
temp, humidityMax: humidity },
        $inc: { count: 1 }
      },
      {
        upsert: true
      }
    )
    sleep(Math.floor(Math.random() * 200));
  }
}
```



Management Stages

- `$collStats`
- `$currentOp`
- `$indexStats`
- `$listLocalSessions`
- `$listSessions`
- `$planCacheStats`

Management stages:

Several commands for collecting statistics and other useful information from the MongoDB server have an aggregation pipeline counterpart, some even only exist as an aggregation pipeline stage. This has the advantage that they can be distributed in a sharded cluster and it makes analysing the resulting data easier.

Usages:

- Use them in a view and then visualize them with MongoDB Charts
- Build your own simple monitoring by writing the results into a collection using the `$merge` stage and then visualize it in MongoDB Charts
- Build an aggregation pipeline to analyse the results of `$planCacheStats` to see which Index is used by which query to identify Indexes that could be removed



Blocking vs Non-Blocking Stages

- A non blocking stage can immediately pass its input document to the next stage in the pipeline in a streaming fashion
Example: `$set`, `$match`, `$unwind`
- Blocking stages need to see all documents before they can send any to the next stage
Examples: `$sort`, `$group`, `$facet`, `$bucket`, `$bucketAuto` and `$sortByCount`

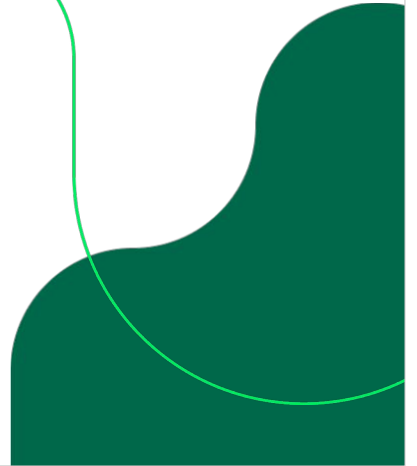
Aggregation stages can be divided into two categories – blocking, and non-blocking.

In a sharded cluster – be aware that as soon as you use a blocking stage you cause all the results to be coalesced onto a single host – for example when sorting the results are sorted on the shards but then merged onto a single host.

For this reason always see if you can find a way to avoid blocking stages. For example, if you want to sort an array inside a document you could use `$unwind`, `$sort` and `$group` by `_id` with `$push` – but this is exceptionally inefficient. `$group` assumes the input may come from multiple documents and so has to wait for every document to complete.

You could instead use the `$set` stage with the `$setUnion` operator and use the empty set `[]` as the second argument for `$setUnion`– MongoDB sorts to compute sets . This is not a documented behaviour and therefore could change in future but is the simplest option currently especially in an ad-hoc aggregation. There is also a longer more complex way which still doesn't use `$group` but uses comprehension as below, we are not listing that here but it is possible to write a full sort function in aggregation.

List Comprehension





List Comprehension Expressions

- \$map

```
{ $set : { squaredlist :  
  { $map : {  
    input: "$some_list",  
    in: { $mul : [ "$$this", "$$this" ]  
  }  
}
```

Besides the \$avg expression there is a whole list of other list comprehension expressions that we can use to work on arrays during projection stages, like \$map, \$reduce, \$zip and \$range.

- \$map is applied to every element in an array, each item is interacted with through \$\$this by default. The result will be an array of values each with the map actions applied.
- \$reduce iterates over an array returning a single value, you specify the original value and the value after examining each list element.



List Comprehension Expressions

- \$map
- \$reduce

```
{ $set : { sumofvals :  
  { $reduce : {  
    input: "$some_list",  
    initialValue: 0,  
    in: { $sum : [ "$$value", "$$this" ]  
  }  
}
```

\$reduce iterates over an array returning a single value, you specify the original value and the value after examining each list element.



List Comprehension Expressions

- `$map`
- `$reduce`
- `$range`
Creates an array of values
- `$zip`
Merge multiple arrays into single array of arrays
`[ [1,2,3], [a,b,c] ] -> [ [1,a], [2,b], [3,c] ]`

- `$zip` takes two or more lists and returns a list of lists where the first list has element zero for each of the inputs, the second list has element one of each of the inputs, and so forth.
- `$range` creates a list/array of values from 1 to n.

These four operations can essentially create a Universal Turing machine and, with effort, solve any problem - they are immensely powerful for computing things inside the database.

Show examples is pages 28, 29 and 30 of the workbook.



Example: Determine if a Number is Prime

Use aggregation to implement an algorithm that determines if a number is prime by taking advantage of list comprehension operators.

We use `$range`, `$map`, and `$reduce` to accomplish our goal, and use `$set` to show the output

First, create a range of documents with values to test:

```
db.numbers.drop()
for(x=0;x<1000;x++) {
  db.numbers.insertOne({_id:x})
}
```

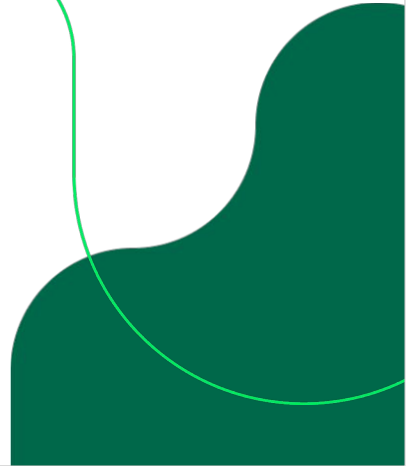
Solution:

Now write an aggregation to tell us which are prime (Ignore 0 and 1), use `$range` to generate a list of factors, `$map` to divide the number by each and `$reduce` to determine if any are an exact division. It's possible to do this without the `$reduce` as well.

```
possible_factors = {
  $range : [ 2,
    {
      $add: [{
        $floor: {
          $sqrt: "$_id"
        }
      }, 1]
    }
  ]
}
```

```
isPrime = {
  $reduce: {
    input : {
      $map : {
        input : possible_factors,
        in: { $divide : ["$_id", "$$this"] }
      }
    },
    initialValue: { $gt: ["$_id", 1] },
    in: {
      $and : [ "$$value", {
        $ne : [ "$$this", {
          $floor: "$$this"
        } ]
      } ]
    }
  }
}
```

Queries and Pipelines in the code



Generating Queries and Pipelines in code

Aggregation pipelines and queries can become very complex
We can write builder functions for them or have variables with reusable parts of an aggregation pipeline

Some uses cases for builder functions:

- Complex aggregation pipelines (and we need to simplify the code)
- Parts of the aggregation pipeline can be reused
- Use data to generate the pipeline
- Clean and create readable code



Generating Queries and Pipelines in code

Challenge:

Temperature collection with documents with Celsius and Fahrenheit degrees

```
{temp: x, unit: "c" }
```

```
{temp: x, unit: "f" }
```

When writing aggregations (or even queries) you want to query for either :

```
db.temperatures.find({ temperature: fahrenheit(451) })
```

Write the function `fahrenheit` that makes the code work and finds either C or F units

Here is a challenge.

Imagine you have a collection which has temperatures as an object. Some are stored as { temp: x, unit: "c" } and some are {temp: x, unit: "f" }

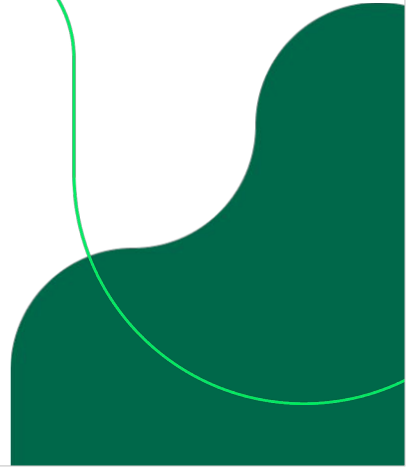
When writing aggregations (or even queries) you want to query for either - write the function `fahrenheit` that makes the following code work. and finds either version.

Remember whatever your language you are never manipulating Javascript strings.

(Discuss solution)

Review the final example about the XOR operations (Workbook page 31)- this code in the workbook returns an expression to XOR two integers - the shift and getbit functions can be easily reused to implement OR and AND. Note: This example is just for demonstration only as there is a XOR stage in the MongoDB aggregation pipeline,

Using \$let





Without \$let

```
values = { $range : [ 0, 360] }

//$sqrt of $a computed 360 times
{ $set : {
  normals : {
    $map : {
      input: values,
      in : {
        $divide : [ "$$this",{$sqrt : "$a"}]
      }
    }
  }
}}
```

In this example \$a is different in each record so we cannot simply hard code the square root of a into our aggregation

Technically we could add it as a temporary field however \$let solves this as a temporary variable.



Using \$let

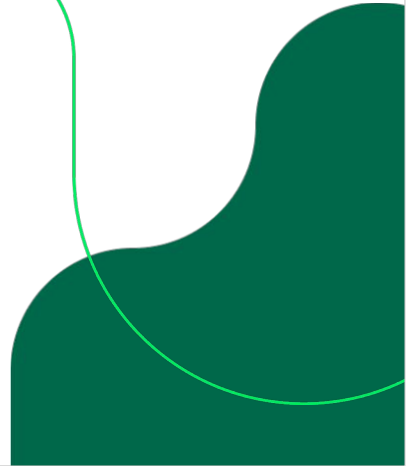
Never recompute what you can precompute

```
// Compute $sqrt of a once and use in the rest as a variable
{$set: {
  normals: {
    $let: {
      vars: { sqrtofa: { $sqrt: "$a" } },
      in: {
        $map: { input: values,
          in: {
            $divide : [ "$$this", "$$sqrtofa" ]
          }
        }
      }
    }
  }
}}}}}
```

In this example \$a is different in each record so we cannot simply hard code the square root of a into our aggregation

Technically we could add it as a temporary field however \$let solves this as a temporary variable.

Updates with Aggregation pipelines





Updates with aggregation pipelines

We can use aggregation pipelines inside an update statement (4.2+)

This allows to perform updates which depend on other fields in the document

Aggregations in updates are limited to setting and unsetting fields to the value of expressions or replacing the entire document

Since MongoDB 4.2 it is also possible to use aggregation pipelines inside an update statement. The aggregation pipeline is then used instead of the update document and can use the \$set stage to set fields in the updated document.

The purpose of this is to be able to perform an update which depends on other fields in the document - before that only \$inc and \$mul were operations that were relative to something else (arguably \$push and \$addToSet)

Aggregations in updates are limited to setting and unsetting fields to the value of expressions or replacing the entire document. You can set one value to the sum of others - or apply conditional logic.



Updates with aggregation pipelines - Exercise:

Write an update to add a flag in the numbers collection saying if the number is prime or not.

Solution (Workbook page 36)

```
possible_factors = {
  $range : [ 2,
    {
      $add: [{
        $floor: {
          $sqrt: "$_id"
        }
      }, 1]
    }
  ]
}
```

```
isPrime = {
  $reduce: {
    input : {
      $map : {
        input : possible_factors,
        in: { $divide : ["$_id", "$$this"]}
      }
    },
    initialValue: {$gt: ["$_id", 1]},
    in: {
      $and : ["$$value", {
        $ne : [ "$$this", {
          $floor: "$$this"
        } ]
      } ]
    }
  }
}
```



Conclusions